Implementation of a Basic Blockchain with PoW and Merkle Patricia Tries

Joel González Jiménez
Universitat Politècnica de Catalunya – FIB
Barcelona, Spain

***Abstract*—Blockchain research is continually driving new functionalities and optimizations. As a result, modern blockchains look vastly different from the beginning. This paper presents a modular, localhost blockchain prototype written in Go, specifically engineered to study the core concepts such as consensus mechanisms, Proof-of-Work (PoW), and cryptografic proofs with modified hexary Merkle Patricia Trie (MPT). By decoupling these pillars from the overhead of peer-to-peer networking, we purpose a solid framework built from scratch for newcomers understand better the foundations of blockchain systems. Also debugging tools and tests are provided to facilitate building on top more functionalities and further optimizations. Crucially, we leverage some changes and systemic constraints of a MPT implementation that utilizes self-referential content-addressed hashing to achieve more efficient space usage.**

***Index Terms*—Blockchain, Merkle Patricia Trie, Proof of Work, Cryptographic Proof.**

I. Introduction

Blockchain research is continually driving new functionalities and optimizations. As a result, modern blockchains look vastly different from the beginning. While early distributed networks relied strictly on simple transaction sequences appended to linear ledgers, contemporary platforms require fine-grained state tracking and highly optimized authenticated storage configurations.

This project is a hands-on implementation of an experimental blockchain environment designed to deliberately isolate and study these core foundational subsystems, focusing comprehensively on:

- **Consensus Mechanism (PoW):** The throttling of ledger updates through intensive algorithmic resource consumption, provided by Bitcoin [1].
- **Merkle Patricia Trie (MPT):** The design of an authenticated, radix-based data organization layer capable of emitting cryptographic proofs, provided by Ethereum [2].

The entire implementation and code can be found available in the open github repository [3].

II. Related Work

The landscape of blockchain research can be overwhelming, to bypass the steep learning curves and noise of complex codebases like Go-Ethereum (Geth), researchers frequently rely on lightweight blockchain frameworks. This frameworks are very useful because can be used to extend functionalities and provide optimizations without making from scratch your own blockchain system.

Minimalist educational implementations, such as *Naivechain* [4] or early Hyperledger prototypes, provide clean environments to study consensus loops but frequently strip away complex world-state tracking, substituting a proper MPT with basic flat arrays or simple cryptographic hash chains. This project bridges this gap by retaining a functionally rigorous, hexary MPT layout and a resource-intensive PoW mining loop, while stripping away the unpredictable latency of P2P network layers to allow direct, deterministic profiling of state integrity.

III. Core System Capabilities

The developed platform provides a robust research testbed that successfully demonstrates four primary system capabilities:

- **Cryptographic Immutability:** Enforcing absolute historical integrity across the ledger via sequential SHA-256 block-hashing tied directly to the execution bounds of the Proof-of-Work consensus loop.
- **Lightweight Verification:** Utilizing a custom Merkle Patricia Trie layout to yield deterministic cryptographic execution proofs, maintaining an efficient operational lookup bounds of $\mathcal{O}(\log n)$. µ
- **Persistent State Storage:** Implementing a reliable underlying data-store layer utilizing a database to preserve the entire blockchain structure across successive experimental runs.
- **Debugging Tools and Tests:** Integrating diagnostic utilities and localized test suites optimized for analyzing the serialization size, runtime growth, and data density of individual block payloads.

IV. Custom Merkle Patricia Trie Design

The data storage architecture modifies the traditional execution parameters of a standard Merkle Patricia Trie to optimize for a per-block data log environment.

A. Architectural Departures from Standard MPT

Unlike generalized global world-state implementations (such as Ethereum), this framework enforces four explicit structural constraints:

1) **GOB Encoding Framework:** The system substitutes standard Recursive Length Prefix (RLP) [2] serialization for Go's native `GOB` [5] binary encoder to natively streamline abstract interface across node variations.

2) **Value-Free Branch Nodes:** Standard MPT branch nodes store a terminating value slot to accommodate variable-length paths. In this framework, **Branch nodes do not contain a value** field because every key in the sub-tree shares a strictly uniform static length constraint.
3) **Self-Referential Content Addressing:** The path/key utilized to navigate to a value inside the trie is uniquely derived as the SHA-256 hash of the target value itself:

$$\text{Path} = \text{SHA256}(\text{Value})$$

Consequently, every key mapping to a node spans exactly 32 bytes (64 hexary nibbles), eliminating path variance.

B. Node Structural Layouts

An example of the structure implemented Merkle Patricia Trie can be seen in Figure 1 . There are three different variations of a `Node`:

- **Leaf Node:** Contains the terminal path slice directly to its corresponding raw data payload.
- **Branch Node:** Is an array of 16 slot hashes (`Childs_hash [16][32]byte`) to manage routing through nibles.
- **Extension Node:** Leverages path compression by squashing a sequence of shared nibbles (`Path_shared`) and contains a reference of the next hash (`Next_hash`).

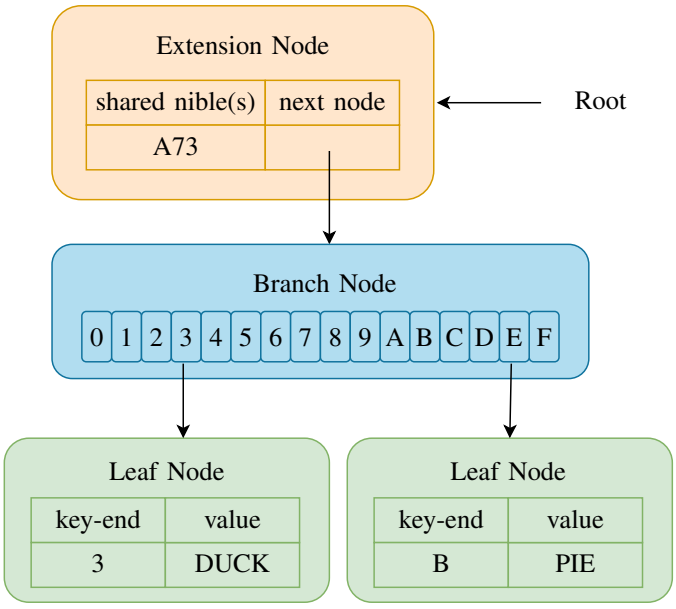

Fig. 1. Custom hexary Merkle Patricia Trie

V. Serialization Analysis: GOB vs. RLP

A foundational design trait of this prototype is its reliance on Go's internal binary encoder (`GOB`) instead of the Recursive Length Prefix (`RLP`).

A. The GOB Rationale

GOB provides exceptional utility inside a specialized, single-language Go prototype. It inherently processes abstract interface schemas. By embedding type descriptions and metadata definitions directly inside the resulting byte array, it allows the storage layer to seamlessly serialize and deserialize abstract `Node` definitions without requiring manual byte-parsing and validation boilerplates.

B. The RLP Consensus Imperative

While optimal for a localized framework, GOB is critically flawed in distributed multi-node consensus environments because it lacks absolute binary determinism. Internal stream optimizations and compiler variant updates can yield subtle discrepancies in output byte arrays from identical input data, causing irreversible ledger forks. Implementing **RLP** represents a vital optimization upgrade for production deployment; RLP omits all struct type schema data, encoding exclusively lengths and raw bytes to ensure absolute cross-client cryptographic determinism.

VI. Consensus Engine: Proof-of-Work

To commit an assembled data block securely to the persistent ledger, the mining module (`pow.go`) forces computational verification.

A. Target Boundary Evaluation

A block's validity is governed by a strict mathematical constraint. The system boundary target big integer T is calculated by executing a left-shift operation against a baseline value based on a globally declared block difficulty variable D:

$$T = 1 \ll (\text{HashLength} - \text{Difficulty}) \tag{1}$$

Where HashLength represents the bit-width of the cryptographic hashing output space (256 bits).

B. The Execution Loop

The consensus engine loops through individual incrementing iterations of a block's 64-bit integer `Nonce` field, validating that the resulting double-pass hash satisfies the inequality:

$$\text{SHA256}(\text{BlockData} \parallel \text{Nonce}) < T \tag{2}$$

Once a nonce is discovered that results in a hash of the block with an integer value below the calculated target line T, the mining cycle terminates, and the block is broadcast to the storage management layer.

VII. Architectural Design

The architectural design strictly prioritizes modularity, enforcing a highly structured, acyclic dependency graph among the application classes to clearly assign operational responsibilities, Figure 2 and 3.

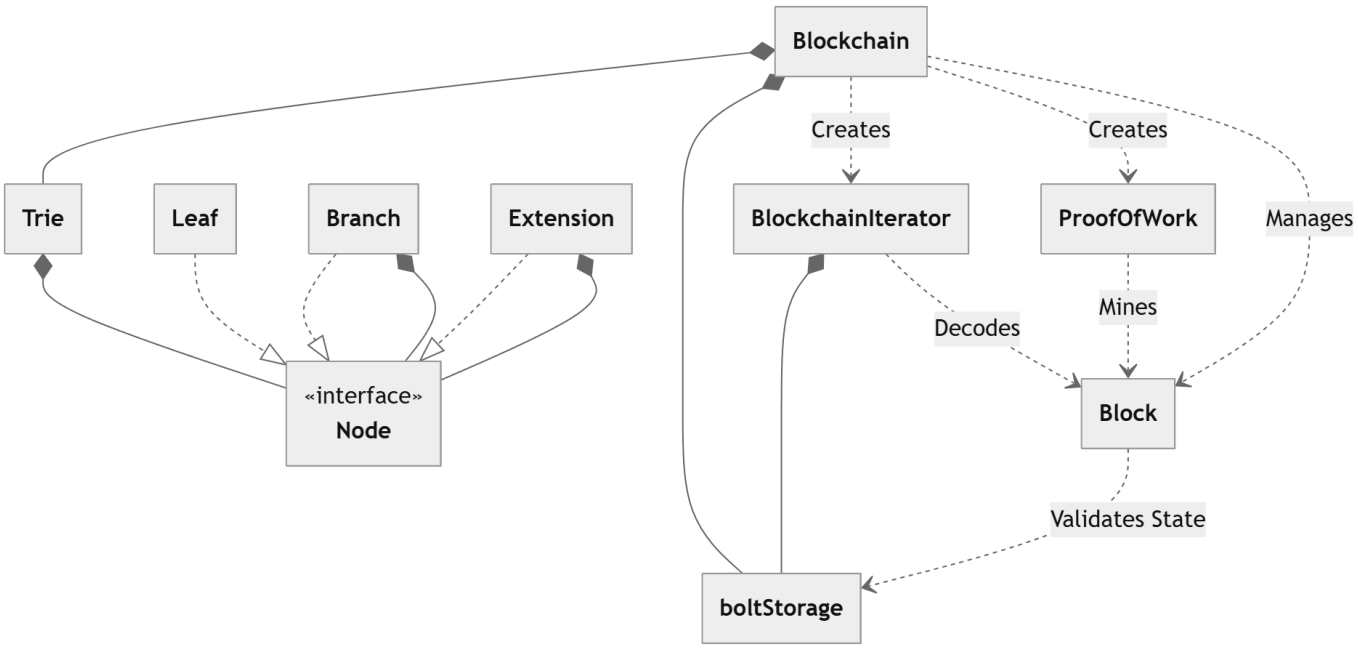

Fig. 2. Class diagram

A. Class Diagram

- **Blockchain.** Acts as the primary entry point, maintaining a direct strong composition relationship over the active `Trie` model and overseeing database transactions.
- **boltStorage.** Utilizes an instance of `boltStorage` to write data into two buckets (`"blocks"` and `"tries"`).
- **BlockchainIterator.** Manages a strong composition relationship back to the database, tracking byte-serialized headers backwards via the block's embedded `PrevBlockHash` hash pointer.
- **Proof of Work.** Responsible of performing the block mining process, trying to achieve a hash lower than a specific target produced by a certain difficulty.
- **Block.** It contains the unique information for identification purpose and also the hash of previous block to grant immutability and enable block iteration over the chain.
- **Trie.** Data structure used to store pair key-values, in this case strings transformed to bytes. The trie only contains the root node, and you can visit all nodes that belong to the trie recursively starting from the root node. A "Node" is a Go interface that could be either a "Leaf", "Branch" or "Extension" classes which functionalities have been explained previously.

A complete version of the class diagram can be found in the github repository [3]

B. System Architecture

The architecture has three main layers which can be well differentiated in the Figure 3. Responsibilities of each layer are the following:

1) **Client Interface.** Is the only part of the program that directly interacts with the user and calls the appropriate functions of the Blockchain core logic layer to perform the actions. This layer is a CLI and has some features like auto-complete and commands usage help.
2) **Blockchain core logic and consensus layer.** Provides the core logic that can perform four main actions:
   - **Chain validation.** Performed every time a block wants to be added to assure that the chain is valid and not a fraud.
   - **Trie creation.** Process of adding elements to a trie till is decided it is completed.
   - **Block assembly.** Once the trie is built a block is initialized with its respective data and the first value of the nonce "0".
   - **Block mining.** Finally, this step performs Proof of Work (PoW), iterating nonce values till it produces a hash value lower than a target. Once the a block has been mined successfully, it is sent to the next layer to be stored.
3) **Persistent storage DB.** Its purpose is to manage the interaction with database (BoltDB) retrieving or storing the information. The database is divided into two buckets for the different kind of data that this program handles (trie and block data).

VIII. Critical Trade-offs and Vulnerabilities

A. Storage Write Amplification

Because the MPT resets cleanly at the termination of each block cycle, the MPT acts as an block isolated ledger rather than a persistent world state. However, during data insertion

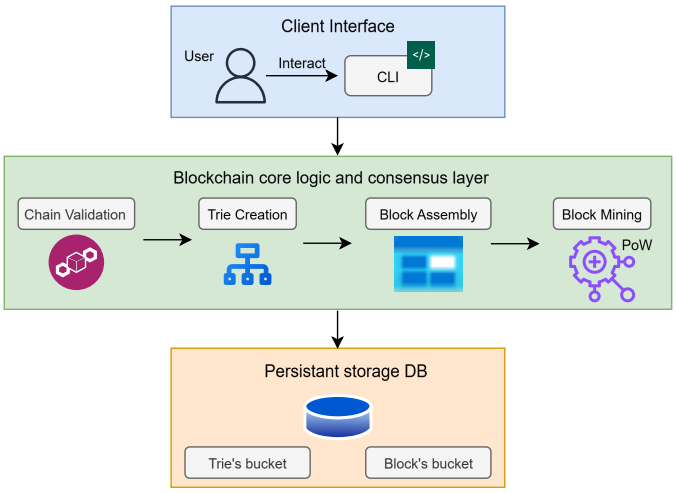

Fig. 3. Architecture

phases, updating a leaf node alters its hash signature, breaking its parent branch hash. This domino effect scales recursively up to the tree head, generating high disk-write workloads proportional to trie depth.

B. Vulnerability Assessment

- **Lack of Authorization and Signature Verification:** Payloads are processed as arbitrary data strings. The missing implementation of public-key cryptography (such as ECDSA or Ed25519 digital signatures) creates a vulnerability where transaction ownership or authenticity cannot be validated.
- **Exposure to State Bloat DoS Attacks:** The absence of an economic gas or processing fee model allows a malicious agent to execute infinite value insertions for free, risking host disk space exhaustion.

IX. Conclusion and Future Work

This paper successfully presented a modular, clean research framework implemented in Go with debugging tools and experimental tests that successfully isolates and studies Proof-of-Work constraints and specialized Merkle Patricia Trie behaviors facilitating newcomers understand better this concepts and extending functionalities. By mapping paths strictly via content hashes, the MPT architecture successfully optimizes out branch value requirements. Future milestones will focus on transitioning this block-centric log structure into a cumulative, rolling world-state engine by persisting root hash variables across block headers, replacing the GOB encoder with a deterministic RLP framework, and establishing cryptographic signature enforcement mechanisms.

References

[1] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” Tech. Rep., 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf
[2] G. Wood, *Ethereum: A Secure Decentralised Generalised Transaction Ledger*, Ethereum Project, 2014, living Technical Specification. [Online]. Available: https://ethereum.github.io/yellowpaper/paper.pdf
[3] J. Ti, “dsproject,” https://github.com/jowy-ti/dsproject, 2026.
[4] L. Hartikka, “lhartikk/naivechain,” May 2026, original-date: 2016-10-09T14:43:06Z. [Online]. Available: https://github.com/lhartikk/naivechain
[5] “gob package - encoding/gob - Go Packages.” [Online]. Available: https://pkg.go.dev/encoding/gob